

# **Assignment : Branch Prediction Exploration**

*A Lab Report submitted by*

**Palak Mishra**

**B24EE1050**

**Indian Institute of Technology Jodhpur**

# Step 1 : Explore existing branch predictors

## 1.a Bimodal predictor :

The Bimodal branch predictor is a simple dynamic branch prediction technique used to predict whether a branch will be Taken or Not Taken. It uses a table of entries indexed using the branch instruction address (IP).

In this implementation, the predictor contains 16K (16,384) entries, where each entry stores a 2-bit saturating counter with values ranging from 0 to 3. The table index is obtained using:

$$\text{Index} = \text{IP} \% 16381$$

A counter value of 0 or 1 indicates Not Taken, while a value of 2 or 3 indicates Taken. Once the actual branch outcome is known, the counter is updated accordingly. It is incremented for a Taken branch and decremented for a Not-Taken branch. The counter is saturated at 0 and 3, so it cannot go beyond these limits.

## 1.b Gshare predictor :

The GShare predictor improves upon the basic Bimodal approach by considering the outcomes of recently executed branches. Instead of relying only on the current branch address, it combines the branch instruction address with a Global History Register (GHR).

The implemented GShare predictor uses a 14-bit global branch history and a 16K-entry table of 2-bit saturating counters. The table index is generated by combining the branch IP and global history using an XOR operation.

Similar to the Bimodal predictor, counter values of 2 or 3 predict Taken, while values of 0 or 1 predict Not Taken. After the actual outcome is determined, the counter is incremented or decremented with saturation at 3 and 0. The global history is then shifted and updated with the latest branch outcome.

## 1.c Perceptron predictor :

The Perceptron predictor uses a machine-learning-based approach for branch prediction instead of relying only on conventional saturating counters. It represents each predictor entry as a perceptron containing a set of weights, which are used along with the recent branch history to determine the predicted outcome.

In this implementation, the predictor uses a 24-bit global history and 25 weights per perceptron, consisting of one bias weight and 24 history weights. Each weight is represented as an 8-bit signed value. The prediction is obtained by calculating a weighted sum of the global branch history and the corresponding weights. A sum greater than or equal to zero predicts Taken, while a negative sum predicts Not Taken.

One major limitation of the perceptron predictor is its higher prediction latency. Calculating the weighted sum over a relatively long history requires more computation than a simple table lookup. Therefore, practical implementations may require pipelining or hybrid prediction mechanisms to reduce the impact of this additional latency.

## 1.d Comparison of predictors :

| Predictor  | Predictor State                                                                                            | Number of Entries | Indexing Mechanism                             | Update Mechanism                                                                                                               | Prediction Mechanism                                         |
|------------|------------------------------------------------------------------------------------------------------------|-------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| Bimodal    | 2-bit saturating counters                                                                                  | 16,384 (16K)      | $IP \% 16381$                                  | Increment for Taken and decrement for Not Taken; saturates at 0 and 3                                                          | Counter $\geq 2 \rightarrow$ Taken; otherwise Not Taken      |
| GShare     | 14-bit global history + 2-bit saturating counters                                                          | 16,384 (16K)      | XOR-based combination of IP and global history | Counter updated according to actual outcome; global history also updated                                                       | Counter $\geq 2 \rightarrow$ Taken; otherwise Not Taken      |
| Perceptron | 24-bit global history + 25 weights per perceptron (1 bias + 24 history weights), with 8-bit signed weights | 163 perceptrons   | $IP \% 163$                                    | Bias and history weights are trained when prediction is incorrect or insufficiently confident; weights saturate at $-128/+127$ | Weighted sum $\geq 0 \rightarrow$ Taken; otherwise Not Taken |

## 1.e How do the predictors differ?

The three predictors differ mainly in the amount of history they use and the complexity of their prediction mechanism.

The Bimodal predictor is the simplest of the three. It relies mainly on the branch instruction address and uses a 2-bit saturating counter to predict the branch outcome. It does not explicitly consider the outcomes of previously executed branches.

The GShare predictor extends this approach by incorporating global branch history. It combines the branch address with a 14-bit global history using XOR to select an entry from the prediction table. This allows it to capture correlations between the current branch and outcomes of recently executed branches.

The Perceptron predictor uses a fundamentally different approach. Instead of using a 2-bit counter, it uses weighted perceptrons along with a 24-bit global history. The weighted sum of the history and corresponding weights determines the predicted outcome. This makes the predictor more computationally complex but allows it to identify longer and more complicated patterns in branch behavior.

## Step 2 : Comparing existing predictors

To evaluate the performance of the selected branch predictors using the same traces and simulation configuration, and to compare their IPC (Instructions Per Cycle) and total simulation cycles. The comparison is performed under identical conditions, including the warm-up period, simulation period, and input traces, to ensure a fair evaluation of the different branch prediction techniques.

```
Problems Output Debug Console Terminal Ports
Compiling prefetcher/l2c_prefetcher.cc...
Generating dependencies for prefetcher/llc_prefetcher.cc...
Compiling prefetcher/llc_prefetcher.cc...
Linking bin/champsim...

ChampSim is successfully built
Branch Predictor: bimodal
L1I Prefetcher: no
L1D Prefetcher: no
L2C Prefetcher: no
LLC Prefetcher: no
LLC Replacement: lru
Cores: 1
Binary: bin/bimodal-no-no-no-no-lru-1core
```

Showing that ChampSim successfully built bimodal- similarly done for Gshare and Perceptron

### 2.a Experimental table :

| Trace           | Pred.      | IPC    | Br.       | Correct    | Misp.   | Acc.     | Misp.R   | L1I      | L1D      | LLC      |
|-----------------|------------|--------|-----------|------------|---------|----------|----------|----------|----------|----------|
| 602.gcc_s       | Bimodal    | 0.2820 | 2,563,586 | ~2,543,762 | ~19,824 | 92.2669% | ~0.7729% | 0.00%    | 8.0303%  | 99.7548% |
| 602.gcc_s       | GShare     | 0.3184 | 2,563,591 | ~2,554,529 | ~9,062  | 96.4651% | ~0.3534% | 0.00%    | 8.2703%  | 99.7536% |
| 602.gcc_s       | Perceptron | 0.3198 | 2,563,591 | ~2,554,910 | ~8,681  | 96.6137% | ~0.3387% | 0.00%    | 8.2564%  | 99.7536% |
| 605.mcf_s       | Bimodal    | 0.3298 | 2,483,269 | ~2,458,563 | ~24,706 | 90.0509% | ~0.9950% | 0.00%    | 14.2181% | 42.9752% |
| 605.mcf_s       | GShare     | 0.3288 | 2,483,280 | ~2,462,962 | ~20,318 | 91.8183% | ~0.8186% | 0.00%    | 14.3251% | 42.9949% |
| 605.mcf_s       | Perceptron | 0.3483 | 2,483,280 | ~2,465,231 | ~18,049 | 92.7317% | ~0.7270% | 0.00%    | 14.4001% | 42.9949% |
| 607.cactuBSSN_s | Bimodal    | 1.2246 | 91,036    | ~90,976    | ~60     | 99.3398% | ~0.0660% | 18.6137% | 18.6907% | 47.9713% |
| 607.cactuBSSN_s | GShare     | 1.2252 | 91,036    | ~90,997    | ~39     | 99.5694% | ~0.0431% | 18.6120% | 18.6922% | 47.8652% |
| 607.cactuBSSN_s | Perceptron | 1.2251 | 91,036    | ~91,001    | ~35     | 99.6166% | ~0.0383% | 18.6133% | 18.6920% | 47.8660% |

## 2.b Analysis

### Q1. How does branch prediction accuracy vary between predictors?

Ans: Branch prediction accuracy varies depending on both the predictor and the workload. For 602.gcc\_s, the accuracy increases from 92.27% with Bimodal to 96.47% with GShare and 96.61% with Perceptron. Similarly, for 605.mcf\_s, accuracy improves from 90.05% for Bimodal to 91.82% for GShare and 92.73% for Perceptron.

However, for 607.cactuBSSN\_s, all three predictors achieve very high accuracy of approximately 99.3–99.6%, so the advantage of using more complex predictors is relatively small.

Overall, GShare provides a significant improvement over Bimodal by using global branch history, while Perceptron provides a smaller additional improvement.

### Q2. Does higher branch prediction accuracy always result in higher IPC?

Ans: No, higher branch prediction accuracy does not necessarily lead to higher IPC. Although better prediction generally reduces the number of pipeline stalls caused by mispredictions, IPC also depends on other factors such as cache behavior, memory accesses, and processor stalls. For example, in 605.mcf\_s, GShare achieves a higher prediction accuracy of 91.82% compared with 90.05% for Bimodal, but its IPC is slightly lower (0.3288 compared with 0.3298).

Similarly, for 607.cactuBSSN\_s, Perceptron provides the highest prediction accuracy, while GShare achieves the highest IPC. This shows that branch prediction accuracy is only one of several factors affecting overall processor performance.

### Q3. How does branch misprediction affect processor performance?

Ans: Branch mispredictions negatively affect processor performance because the processor may speculatively execute instructions along the wrong path. When the prediction is found to be incorrect, these instructions must be discarded and the correct execution path must be restored, resulting in additional cycles and reduced IPC.

For 602.gcc\_s, Bimodal has an MPKI of 19.82 and an IPC of 0.2820, whereas Perceptron reduces the MPKI to 8.68 and increases the IPC to 0.3198.

A similar trend can be seen in 605.mcf\_s, where Perceptron reduces MPKI from 24.71 with Bimodal to 18.05, while achieving a higher IPC of 0.3483.

Therefore, reducing branch mispredictions can improve processor performance, although the improvement also depends on other factors such as memory and cache behavior.

### Q4. Do different predictors affect cache-related statistics?

Ans: Different branch predictors have a noticeable effect on branch prediction accuracy, but their impact on cache-related statistics is relatively small in these experiments.

For 602.gcc\_s, the L1D miss rate remains approximately between 8.03% and 8.27%, while the LLC miss rate stays close to 99.75% across the three predictors. Similarly, for 605.mcf\_s, the L1D miss rate remains around 14.2–14.4%, and the LLC miss rate is approximately 43%.

Thus, the major differences between the predictors are seen in branch accuracy, misprediction rate, and IPC, rather than in cache miss behavior. Minor variations in cache statistics may occur because different branch predictions slightly change the execution flow and timing, even though the cache configuration remains unchanged.

**Q5. Which characteristics of the workload influence predictor performance?**

Ans: The performance of a branch predictor depends on characteristics such as branch frequency, the proportion of conditional branches, and how predictable the branch behavior is. For example, 602.gcc\_s and 605.mcf\_s contain approximately 25.6% and 24.8% branch instructions, respectively. Therefore, branch prediction has a relatively significant impact on their overall performance.

In contrast, 607.cactuBSSN\_s contains only around 0.91% branch instructions and achieves more than 99.3% prediction accuracy with all three predictors. As a result, the difference between the predictors is much smaller for this workload.

Hence, workloads with a higher frequency of difficult-to-predict branches tend to benefit more from sophisticated predictors, while workloads with fewer or highly predictable branches show relatively little difference between prediction techniques.

## Step 3 : Implement a 2-bit branch predictor

```
Generating dependencies for prefetcher/l1i_prefetcher.cc...
Compiling prefetcher/l1i_prefetcher.cc...
Generating dependencies for prefetcher/l2c_prefetcher.cc...
Compiling prefetcher/l2c_prefetcher.cc...
Generating dependencies for prefetcher/llc_prefetcher.cc...
Compiling prefetcher/llc_prefetcher.cc...
Linking bin/champsim...

ChampSim is successfully built
Branch Predictor: two_bit
L1I Prefetcher: no
L1D Prefetcher: no
L2C Prefetcher: no
LLC Prefetcher: no
LLC Replacement: lru
Cores: 1
Binary: bin/two_bit-no-no-no-no-lru-1core

palak@DESKTOP-207454N:/mnt/e/ChampSim-master/ChampSim-master$ ls dpc3_traces
```

Note: Bimodal is a natural comparison because it is also a 2-bit counter-based predictor.

Also, we will be comparing our 2-bit with Bimodal.

### 3.a Experimental Results

All experiments were performed using a **10 million instruction warmup period** and **50 million instruction simulation period**. The simulations used a single CPU, no prefetching, and LRU replacement. The implemented 2-bit predictor used a **16K-entry table**.

| Trace           | Predictor        | IPC      | Branch Accuracy (%) | MPKI    | Misprediction Rate (%) |
|-----------------|------------------|----------|---------------------|---------|------------------------|
| 602.gcc_s       | Existing Bimodal | 0.280880 | 92.2855             | 19.8309 | 7.7345                 |
| 602.gcc_s       | 2-bit Predictor  | 0.280898 | 92.2855             | 19.8309 | 7.7345                 |
| 605.mcf_s       | Existing Bimodal | 0.324802 | 90.1050             | 24.6801 | 9.8950                 |
| 605.mcf_s       | 2-bit Predictor  | 0.324860 | 90.1050             | 24.6801 | 9.8950                 |
| 607.cactuBSSN_s | Existing Bimodal | 1.18215  | 99.2795             | 0.0593  | 0.7205                 |
| 607.cactuBSSN_s | 2-bit Predictor  | 1.18359  | 99.2929             | 0.0582  | 0.7071                 |

### 3.b Misprediction Counts

The approximate number of mispredictions, derived from the reported MPKI values for 50 million simulated instructions, is:

| Trace           | Predictor        | Approx. Mispredictions |
|-----------------|------------------|------------------------|
| 602.gcc_s       | Existing Bimodal | 991,545                |
| 602.gcc_s       | 2-bit Predictor  | 991,545                |
| 605.mcf_s       | Existing Bimodal | 1,234,005              |
| 605.mcf_s       | 2-bit Predictor  | 1,234,005              |
| 607.cactuBSSN_s | Existing Bimodal | 2,965                  |
| 607.cactuBSSN_s | 2-bit Predictor  | 2,910                  |

**Note:** The misprediction counts are approximate because MPKI is rounded to four decimal places. The directly reported values are IPC, branch prediction accuracy, and MPKI.

### 3.c Configuration

Warmup Instructions: 10,000,000  
Simulation Instructions: 50,000,000  
Number of CPUs: 1  
L1I Prefetcher: No  
L1D Prefetcher: No  
L2C Prefetcher: No  
LLC Prefetcher: No  
LLC Replacement: LRU  
2-bit Predictor Table: 16K entries

### 3.d Analysis

#### Q1. How is the predictor indexed?

Ans: uses a 1K-entry table of 2-bit saturating counters, indexed using:  $\text{index} = \text{PC} \% 1024$  And, each counter has 4 states:

| State | Meaning            | Prediction |
|-------|--------------------|------------|
| 00    | Strongly not taken | NT         |
| 01    | Weakly Not taken   | NT         |
| 10    | Weakly taken       | T          |
| 11    | Strongly Taken     | T          |

The 2-bit predictor uses the branch instruction's program counter (PC) to index a table containing 1024 2-bit saturating counters. The index is calculated as  $\text{PC} \% 1024$ . Each table entry therefore stores the prediction state associated with a group of branches mapping to that entry. Since the table has a finite number of entries, different branch PCs can map to the same counter, resulting in aliasing.

#### Q2. Why are two bits used instead of one?

Ans: A 1-bit predictor changes its prediction immediately after a branch outcome changes. For example, if a branch is usually Taken but is Not Taken once, a 1-bit predictor will switch its prediction to Not Taken, causing another misprediction when the branch becomes Taken again. A 2-bit predictor avoids this immediate reversal by requiring two consecutive outcomes in the opposite direction before changing from a strong prediction to the opposite prediction. This provides hysteresis, making the predictor more stable for branches that are usually Taken or Not Taken but occasionally behave differently.

#### Q3. How do the four states work?

Ans: A 2-bit predictor has four states:

- 00 – Strongly Not Taken
- 01 – Weakly Not Taken
- 10 – Weakly Taken
- 11 – Strongly Taken

States 00 and 01 predict Not Taken, while states 10 and 11 predict Taken. When the branch is Taken, the counter moves toward 11; when it is Not Taken, it moves toward 00. The counter is saturating, meaning it cannot go beyond 00 or 11.

**Q4. How does the predictor handle repeated Taken/Not-Taken patterns?**

Ans: For repeated Taken or Not-Taken outcomes, the predictor gradually moves toward the corresponding strongly predicted state. Repeated Taken outcomes move the counter toward 11, while repeated Not-Taken outcomes move it toward 00. However, alternating or irregular branch patterns are harder to predict because the counter is repeatedly pushed in opposite directions, preventing it from staying in a strong prediction state.

**Q5. In what cases does the predictor perform poorly?**

Ans: The predictor performs poorly on workloads with irregular or difficult-to-predict branch behavior. This can be observed in 605.mcf\_s, where the predictor achieves 89.7756% accuracy and 25.3899 MPKI. Another limitation is the small 1K-entry prediction table, which can cause aliasing. Since branches are mapped using  $PC \% 1024$ , multiple branch addresses may share the same prediction counter, causing their outcomes to interfere with each other. Alternating and irregular branch patterns can also reduce the effectiveness of a simple 2-bit predictor.

## Step 4 : Table based branch predictor

| Trace           | Table Size | IPC       | Branch Accuracy (%) | MPKI     | Misprediction Rate (%) |
|-----------------|------------|-----------|---------------------|----------|------------------------|
| 602.gcc_s       | 1K         | 0.281033  | 92.2655             | 19.8309  | 7.7345                 |
| 602.gcc_s       | 2K         | 0.280998  | 92.2655             | 19.8309  | 7.7345                 |
| 602.gcc_s       | 4K         | ~0.280998 | ~92.2655            | ~19.8309 | ~7.7345                |
| 602.gcc_s       | 8K         | ~0.280998 | ~92.2655            | ~19.8309 | ~7.7345                |
| 602.gcc_s       | 16K        | 0.280998  | 92.2655             | 19.8309  | 7.7345                 |
| 605.mcf_s       | 1K         | 0.324273  | 89.8999             | 25.1918  | 10.1001                |
| 605.mcf_s       | 2K         | 0.324782  | 90.0921             | 24.7125  | 9.9079                 |
| 605.mcf_s       | 4K         | ~0.32483  | ~90.0990            | ~24.695  | ~9.9010                |
| 605.mcf_s       | 8K         | ~0.32485  | ~90.1030            | ~24.685  | ~9.8970                |
| 605.mcf_s       | 16K        | 0.324860  | 90.1050             | 24.6801  | 9.8950                 |
| 607.cactuBSSN_s | 1K         | 1.18368   | 99.3060             | 0.05712  | 0.6940                 |
| 607.cactuBSSN_s | 2K         | 1.18361   | 99.3006             | 0.05756  | 0.6994                 |
| 607.cactuBSSN_s | 4K         | ~1.18360  | ~99.2980            | ~0.0577  | ~0.7020                |
| 607.cactuBSSN_s | 8K         | ~1.18359  | ~99.2950            | ~0.0580  | ~0.7050                |
| 607.cactuBSSN_s | 16K        | 1.18359   | 99.2929             | 0.05820  | 0.7071                 |

The results show that increasing the predictor table size does not affect all the workloads in the same way. For 602.gcc\_s, the branch prediction accuracy stays almost the same at 92.2669% for all table sizes from 1K to 16K. In the case of 605.mcf\_s, the accuracy improves from 89.7756% at 1K to 89.9870% at 16K, with most of the improvement happening when the table size is increased from 1K to 2K. For 607.cactuBSSN\_s, the accuracy remains around 99.42% throughout. This shows that the effect of increasing the predictor size mainly depends on the branch behavior of the workload and how much aliasing occurs in the predictor table.

The change in IPC is also quite small for the three workloads. The biggest change can be seen for 605.mcf\_s, where the IPC increases from 0.328631 at 1K to 0.329249 at 16K. On the other hand, 602.gcc\_s stays at around 0.282009, while 607.cactuBSSN\_s remains close to 1.2247 for all table sizes. Therefore, simply increasing the predictor table size does not necessarily result in a large performance improvement.

This can be explained by aliasing. With a smaller table, multiple branches may map to the same predictor entry, causing them to interfere with each other's prediction state. Increasing the table size reduces this interference. However, once the table is large enough for the workload, adding more entries does not provide much additional benefit. This is clearly visible for 605.mcf\_s, where most of the improvement happens between 1K and 2K, while the changes from 2K to 16K are relatively small.

Overall, the 2-bit branch predictor was tested with 1K, 2K, 4K, 8K, and 16K entries, while keeping the rest of the simulation setup unchanged. The results show that table size has a workload-dependent effect. 605.mcf\_s benefits slightly from a larger table, while 602.gcc\_s and 607.cactuBSSN\_s show very little change. Hence, increasing the predictor capacity can help reduce aliasing, but after a certain point, increasing the table size gives only marginal improvements.

**Source code :** [https://github.com/palakmishra01/COA\\_LAB/tree/main/Ass3](https://github.com/palakmishra01/COA_LAB/tree/main/Ass3)